

Do you remember when prompt engineer was the hot new profession? Prompt engineers could

whisper the right combination of magic words to a large language model to get them to do things

that regular folks issuing regular prompts simply couldn't do. Well, as LLMs got smarter and better

understanding intent, the title of prompt engineer has lost some of its shine. But the fact remains

that the output of LLM is not predictable. LLMs don't behave like deterministic functions like

most other things in computing. They're actually probabilistic. Each token is sampled from a

distribution conditioned on everything that came before. Change the wording a little bit or add

another example or change the temperature and well, you can end up with a different response.

And in chat, maybe that's fine. In software, could be a bit of a bug factory. So, let me give you an example of what I mean. So, let's use an LLM to structure bug reports. I'll supply the

bug report in free form text. And I want the LLM to return strict JSON with this shape. So,

we've got a summary string here. We've got a severity string of either low or medium or high and then we have got a series of steps. Now if I use a chatbot interface or an API call

to invoke an LLM, I can include some instructions in my prompt. So it might be something like you are

a triage assistant return JSON with this format. Here's the first bug report that goes to the LLM

and well it might do it. In fact, most of the time probably will. But every now and again, an LLM might not quite follow the path. It might not return just the JSON at all. Or perhaps it

wraps the JSON in a friendly sentence like, "Sure, here's the reformatted report." Or maybe

it drifts off schema, so it renames summary as synopsis. Well, when software is expecting precise

JSON like this in a precise format and it gets all these variances, well, that's when things start to

break. So, to somebody working to incorporate LLM output into software, prompt engineering actually

means a few very specific things. One of those is contract, which is where the shape of the output

is decided up front, like which keys and enums to use. It also means defining a control loop to

validate every response against the contract and if it fails to automatically retry with tighter

instructions or a constrained decoder and it also means being observable. So

observability. So for example tracing so you can see exactly which prompts produced what. So changes don't ship unless the numbers say they're safe. So today there are tools that can help with that form of prompt engineering and we're going to take a look at two PDL but first of all langchain. So langchain is an open- source framework for building LLM apps with a pipeline of composable steps. So you define what happens before and after a model call not just the words that you send to it. So let's use our triage to JSON example. So at the very top of this example, we need something to send to the model. So we are going to have our user bug text as our input and we're going to send that into the element called a prompt template that will receive the user bug text. Now in lang chain, each box here is a runnable. That's a step that takes some input, does something, and then outputs a result. So the prompt template runnable packages the instruction, the prompt like you're a triage assistant, output JSON only. Here's the shape of the JSON. It will combine that with the user bug text and the same template is reused each time so the wording stays consistent. Now that gets sent to a chat model, the actual LLM itself. So this is a chat model runnable and that will call an LLM and produce a response. So let's call this response the candidate JSON that we've received back from the LLM as a text string. Now next we go to another runnable. This one is called the validate runnable and that checks the candidate response against our schema checking if things like the keys are present and the steps in error a non-empty list and so forth. And if that passes validation then it ends up getting sent to our overall application. So this is the application that is going to receive all of this JSON that we actually want to send it to. So that's if it works. If it fails, we go down a different path where we go to uh another runnable called retry or repair. So, this is the fail path. Now, this is a runnable that can retry, send another model message with some firmware instructions or it can repair, it can make a small fix like strip out the extra pros that came with the JSON. Now, if that passes validation, then it's all good. It's off to the application we go. If that still doesn't work,

then we take what's called a fallback path. And for example, there we might try a stricter model.

And whether it passes first try or we need to do a retry and repair or we need to go through

to the fallback eventually the app receives clean JSON and we keep the traces and the metrics around

as well so we can spot regressions and improve over time. So that's langchain. What about PDL?

That's Prompt Declaration Language. Now, PDL is a declarative spec for LLM workflows. And

the core idea is that most LLM LLM interactions really about producing data. So, you declare the

shape of that data and the steps to produce it. And you do that all within a single file, a YAML

file to be precise. And then a PDL interpreter runs that file. It assembles context. It calls

models and tools. It enforces types and it emits results. So with the PDL in this file here,

we've got our prompt which is defined the thing that we're going to call. We've got the contract

which is what we want this to actually produce. And then we've got the control loop and they all

live in this one file, this YAML file. Now a bit more about the PDL spec itself. So the top level

text is an ordered list where each item can be either a literal string or it can be a block that

calls out to a model. PDL runs this list top down. So strings are appended to the running output and

to a background check context. So when it hits a model block, the default input is everything

that's produced so far unless you provide an explicit input. Now you can declare types in PDL

as well. Those are types for input and output on model steps. So the interpreter does type

checking and it fails on shape violations. Control is explicit. We have things like conditionals for

control and we also have loops we can add in for control. And typical programs also use depths.

Those are used to read data like reading data from a file or from standard in. And then there is a

final data section to collect the name results you want to emit. tracing and live explorer let

you inspect each block's inputs and outputs and the exact context that was sent to the model.

So basically we have got langchain that we've talked about today and what langchain really

is it's something that can be considered code first. It's a code first pipeline with runnables and you wire those runnables together. PDL on the other hand, PDL is really spec first, everything lives in this YAML file where the prompt and the types and the control flow live together and are executed by an interpreter. And together, tools like these are really becoming the grownup toolbox that are turning all of this prompt whispering into real software engineering.